What is Seaborn?

```
Start coding or generate with AI.
```

Seaborn is a Python library used for data visualization. It is built on top of Matplotlib and provides an easier way to create beautiful, attractive, and informative statistical graphs.

Developed by: Michael Waskom

Features of Seaborn

Easy to create attractive graphs Built on Matplotlib Works directly with Pandas DataFrames Provides statistical visualizations Built-in themes and color palettes Less code compared to Matplotlib Supports categorical and numerical data visualization

Advantages of Seaborn

Easy to learn and use Attractive default graph styles Better statistical plotting Works well with NumPy and Pandas Built-in datasets for practice Less coding required than Matplotlib

Disadvantages of Seaborn

Less flexible than Matplotlib for detailed customization Mostly designed for statistical visualization Depends on Matplotlib to display plots

```
pip install seaborn
```

Import Seaborn

```
import seaborn as sns
import matplotlib.pyplot as plt
```

Check Version

```
import seaborn as sns

print(sns.__version__)
```

import seaborn as sns

print(sns.**version**)

```
import seaborn as sns

df = sns.load_dataset("tips")

print(df.head())
```

Viewing Data

```
df.head()

df.tail()

df.info()

df.describe()
```

Basic Line Plot

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset("tips")

sns.lineplot(x="total_bill", y="tip", data=tips)

plt.show()
```

Scatter Plot

```
sns.scatterplot(
x="total_bill",
y="tip",
data=tips
)

plt.show()
```

Bar Plot

```
sns.barplot(
x="day",
y="total_bill",
data=tips
)

plt.show()
```

Use: Compares average values across categories.

Histogram

```
sns.histplot(
data=tips,
x="total_bill"
)

plt.show()
```

use Shows the distribution of numerical data.

KDE Plot

```
sns.kdeplot(
data=tips,
x="total_bill"
)

plt.show()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_791/2395073194.py in <cell line: 0>()
----> 1 sns.kdeplot(
      2 data=tips,
      3 x="total_bill"
      4 )
      5

NameError: name 'sns' is not defined
```

Use: Displays a smooth estimate of the data distribution

Box Plot

```
sns.boxplot(
x="day",
y="total_bill",
data=tips
)

plt.show()
```

Use: Shows data spread, median, and possible outliers.

Violin Plot

```
Start coding or generate with AI.
```

```
sns.violinplot(
x="day",
y="total_bill",
data=tips
)

plt.show()
```

Use: Shows both the distribution and density of data.

Strip Plot

```
sns.stripplot(
x="day",
y="total_bill",
data=tips
)

plt.show()
```

Use: Displays individual data points.

Swarm Plot

```
sns.swarmplot(
x="day",
y="total_bill",
data=tips
)

plt.show()
```

Use: Displays data points without overlapping.

Pair Plot

```
sns.pairplot(tips)

plt.show()
```

Use: Shows relationships between all numerical columns.

Heatmap

```
corr = tips.corr(numeric_only=True)

sns.heatmap(corr)

plt.show()
```

Use: Displays correlation between variables.

Correlation Heatmap

```
sns.heatmap(
corr,
annot=True
)

plt.show()
```

Regression Plot

```
sns.regplot(
x="total_bill",
y="tip",
data=tips
)

plt.show()
```

Use: Shows the relationship between variables with a best-fit line.

Joint Plot

```
sns.jointplot(
x="total_bill",
y="tip",
data=tips
)
```

Distribution Plot

```
sns.displot(
tips["total_bill"]
)
```

Facet Grid

```
g = sns.FacetGrid(
tips,
col="sex"
)

g.map(
plt.hist,
"total_bill"
)
```

Styling

Theme

```
sns.set_theme()
```

Dark Theme

```
sns.set_style("dark")
```

White Grid

```
sns.set_style("whitegrid")
```

Dark Grid

```
sns.set_style("darkgrid")
```

White

```
sns.set_style("white")
```

Ticks

```
sns.set_style("ticks")
```

Color Palette

```
sns.color_palette()
```

Set Palette

```
sns.set_palette("deep")
```

Figure Size

```
plt.figure(figsize=(8,5))
```

Title

```
plt.title("Sales Report")
```

X Label

```
plt.xlabel("Month")
```

Y Label

```
plt.ylabel("Sales")
```

Save Graph

```
plt.savefig("graph.png")
```

Show Graph

```
plt.show()
```

Seaborn with Pandas

```
import pandas as pd
import seaborn as sns

data = {
"Marks":[80,90,70,95]
}

df = pd.DataFrame(data)

sns.histplot(df["Marks"])

plt.show()
```

Seaborn with NumPy

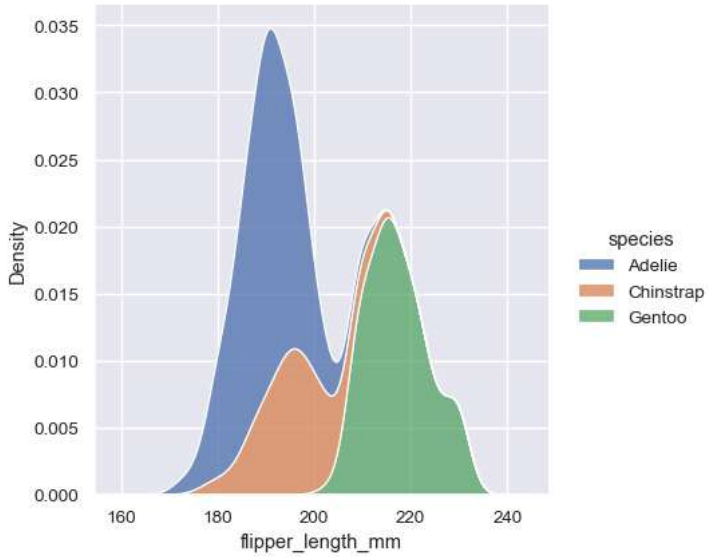
```
import numpy as np

data = np.random.randn(100)

sns.histplot(data)

plt.show()
```

Common Seaborn Functions

Function Purpose lineplot() Draw line graph scatterplot() Draw scatter plot barplot() Draw bar chart countplot() Count categories histplot() Histogram kdeplot() Density plot boxplot() Box plot violinplot() Violin plot stripplot() Strip plot swarmplot() Swarm plot pairplot() Pairwise relationships heatmap() Correlation heatmap regplot() Regression plot jointplot() Joint distribution displot() Distribution plot FacetGrid() Multiple plots

Applications of Seaborn

Data visualization Statistical analysis Data science Machine learning Artificial Intelligence Business analytics Financial analysis Scientific research Medical data analysis Educational projects



Conclusion

Seaborn is a powerful Python library for statistical data visualization. Built on top of Matplotlib, it makes it easy to create beautiful and informative graphs with less code. It integrates seamlessly with Pandas and NumPy, making it an essential tool for data analysis, machine learning, artificial intelligence, business analytics, and scientific research. Its attractive default styles and rich collection of statistical plots make it one of the most popular visualization libraries in Python.

Double-click (or enter) to edit

Double-click (or enter) to edit